

Experiment No. 5

“Implementation of Constructors, Destructors and Multiple Inheritance in C++”

1. Aim

To write a C++ program to illustrate the use of **default constructor**, **parameterized constructor**, and **copy constructor**. Also, demonstrate the order of execution of constructors and destructors when a new class is derived from more than one base class using multiple inheritance.

2. Learning Outcomes

After completing this experiment, students will be able to:

1. Understand the concept of constructors and destructors in C++.
2. Differentiate between default, parameterized, and copy constructors.
3. Implement constructor overloading in C++.
4. Understand the working of destructors.
5. Implement multiple inheritance using C++ classes.
6. Observe the order of execution of constructors and destructors.
7. Apply object-oriented concepts for efficient program design.
8. Develop reusable and maintainable C++ programs using constructors and inheritance.

3. Prerequisites

Students should have knowledge of:

- Basic C++ Programming
- Classes and Objects
- Member Functions
- Constructors
- Destructor
- Inheritance
- Multiple Inheritance
- Object-Oriented Programming Concepts

4. Software / Tools Required

1. Operating System : Ubuntu Linux
2. Programming Language : C++
3. Compiler : GNU G++
4. Text Editor : Gedit / VS Code
5. Terminal : Ubuntu Terminal

5. Theory

Constructors and destructors are special member functions of a class that are automatically invoked during the creation and destruction of objects. Constructors initialize the data members of an object, while destructors release resources occupied by the object before it is destroyed. C++ provides different types of constructors such as default constructors, parameterized constructors, and copy constructors. In addition, multiple inheritance allows a derived class to inherit properties from more than one base class. The order in which constructors and destructors are executed is determined by the inheritance hierarchy.

5.1 Constructor

A constructor is a special member function of a class that is automatically called whenever an object of that class is created. It has the same name as the class and does not have any return type.

Characteristics

- Automatically invoked when an object is created.
- Used to initialize data members.
- Has the same name as the class.
- Does not return any value.
- Can be overloaded.

General Syntax

```
class Demo
{
public:
    Demo()
    {
        // Constructor
    }
};
```

5.2 Default Constructor

A default constructor is a constructor that does not take any arguments. It initializes an object with default values assigned by the programmer.

Features

- Takes no parameters.
- Called automatically when an object is created without arguments.
- Used for default initialization.

Example

```
class Student
{
public:
    Student()
    {
        cout<<"Default Constructor";
    }
}
```

```
};
```

5.3 Parameterized Constructor

A parameterized constructor accepts one or more arguments and initializes the object using the values supplied by the user.

Features

- Accepts parameters.
- Provides customized initialization.
- Can be overloaded.

Example

```
class Student
{
public:
    Student(int r)
    {
        cout<<"Roll No : "<<r;
    }
};
```

5.4 Copy Constructor

A copy constructor initializes one object using another existing object of the same class.

It is automatically called when:

- One object is initialized using another object.
- An object is passed by value.
- An object is returned by value.

Syntax

```
class Demo
{
public:
    Demo(Demo &obj)
    {
        // Copy Constructor
    }
};
```

Example

```
Student s1(10);
Student s2(s1);
```

5.5 Destructor

A destructor is a special member function that is automatically executed when an object goes out of scope or is destroyed.

It performs cleanup operations such as releasing allocated memory and closing files.

Characteristics

- Same name as the class preceded by '~'
- No arguments
- No return type
- Only one destructor per class

Syntax

```
class Demo
{
public:
    ~Demo()
    {
        // Destructor
    }
};
```

5.6 Multiple Inheritance

Multiple inheritance allows a class to inherit data members and member functions from more than one base class.

It enables combining features of multiple classes into a single derived class.

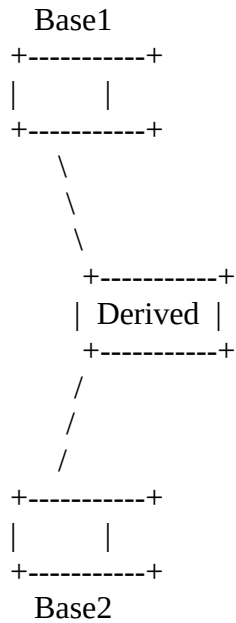
Syntax

```
class Base1
{
};

class Base2
{
};

class Derived : public Base1, public Base2
{
};
```

Class Diagram

**5.7 Order of Constructor and Destructor Execution**

When an object of a derived class is created using multiple inheritance, constructors are executed in the following order:

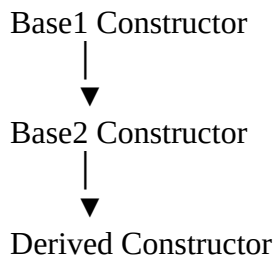
1. Constructor of Base Class 1
2. Constructor of Base Class 2
3. Constructor of Derived Class

When the object is destroyed, destructors are executed in the reverse order:

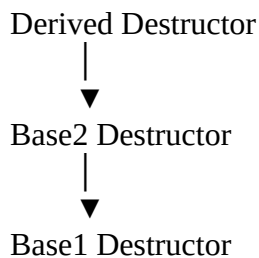
1. Destructor of Derived Class
2. Destructor of Base Class 2
3. Destructor of Base Class 1

Execution Diagram

Object Creation



Object Destruction



5.8 Applications

Constructors, destructors, and multiple inheritance are widely used in software development for efficient object initialization and resource management.

Applications include:

- Banking Management System
- Student Information System
- Employee Payroll System
- Library Management System
- Hospital Management System
- Inventory Control System
- Hotel Reservation System
- Airline Reservation System
- Vehicle Management System
- Real-Time Embedded Systems

6. Algorithm

The following algorithm explains the implementation of **default constructor, parameterized constructor, copy constructor, destructor, and multiple inheritance** in C++. The program demonstrates the automatic execution order of constructors during object creation and destructors during object destruction.

Algorithm Description

Step 1: Start the program.

Step 2: Include the required header files.

Step 3: Create the first base class (**Base1**) and define its default constructor and destructor.

Step 4: Create the second base class (**Base2**) and define its default constructor and destructor.

Step 5: Create the derived class (**Derived**) using multiple inheritance from Base1 and Base2.

Step 6: Define the default constructor, parameterized constructor, copy constructor, and destructor inside the Derived class.

Step 7: Create the first object using the default constructor.

Step 8: Create the second object using the parameterized constructor.

Step 9: Create the third object using the copy constructor.

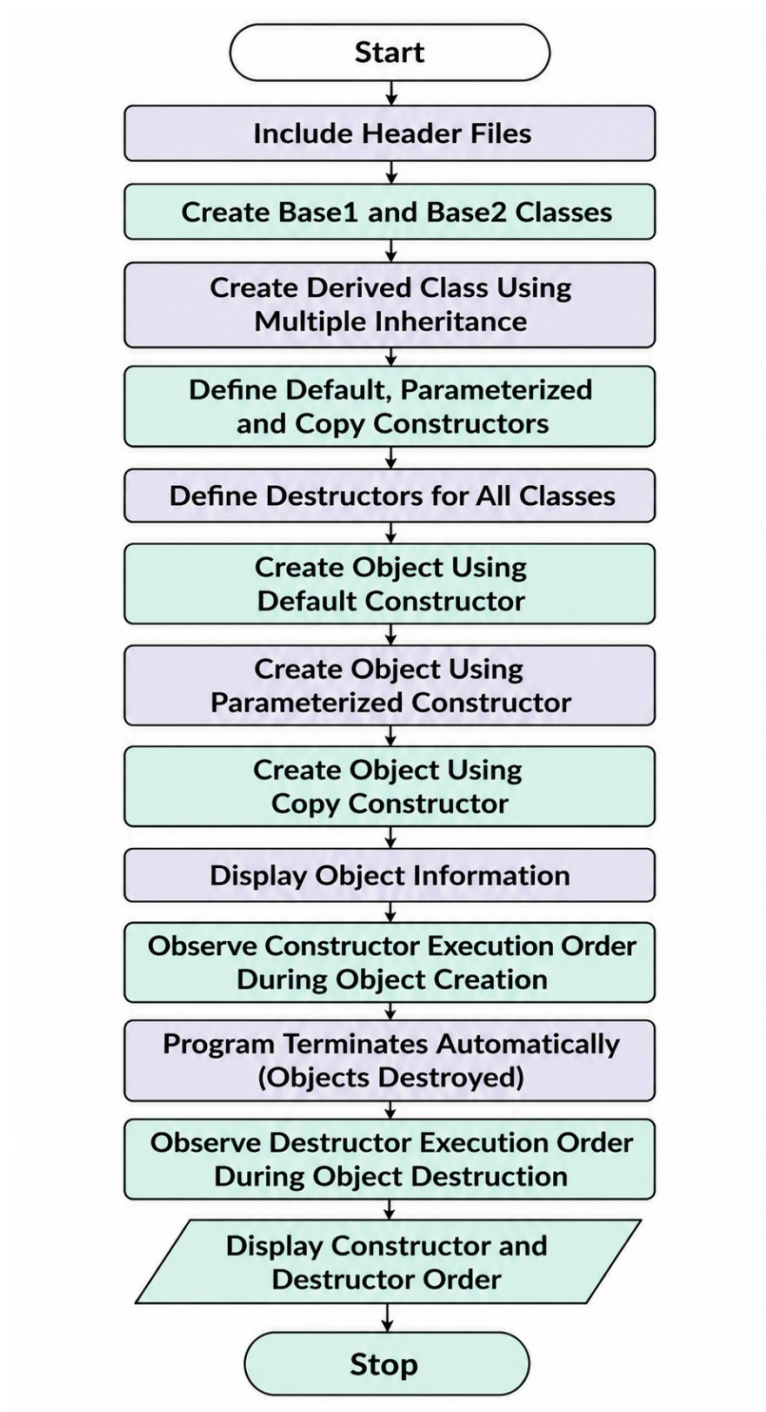
Step 10: Display the values stored in each object.

Step 11: Observe the order in which constructors are executed.

Step 12: Terminate the program and observe the order in which destructors are executed automatically.

Step 13: Stop the program.

The flowchart shown below illustrates the overall execution flow of the program.



7. Lab Exercise

Perform the following exercises to understand constructors, destructors, and multiple inheritance.

Exercise 1: Create Base Classes

Create two base classes:

- Base1
- Base2

Implement constructors and destructors for both classes.

Exercise 2: Create Derived Class

Create a derived class using multiple inheritance.

```
class Derived : public Base1, public Base2
{
};
```

Exercise 3: Implement Default Constructor

Create a default constructor for the derived class.

Display a suitable message whenever it is executed.

Exercise 4: Implement Parameterized Constructor

Create a parameterized constructor that initializes data members.

Display initialized values.

Exercise 5: Implement Copy Constructor

Create a copy constructor.

Copy the values from one object to another.

Exercise 6: Implement Destructor

Create destructors for all classes.

Display appropriate messages to observe their execution order.

Exercise 7: Execute the Program

Create three objects:

- Object using Default Constructor
- Object using Parameterized Constructor
- Object using Copy Constructor

Observe:

- Constructor execution order
- Destructor execution order

8. Program

```
#include<iostream>
using namespace std;

class Base1
{
public:
    Base1()
    {
        cout<<"Base1 Constructor Called"<<endl;
    }

    ~Base1()
    {
        cout<<"Base1 Destructor Called"<<endl;
    }
};

class Base2
{
public:
    Base2()
    {
        cout<<"Base2 Constructor Called"<<endl;
    }

    ~Base2()
    {
        cout<<"Base2 Destructor Called"<<endl;
    }
};

class Derived : public Base1, public Base2
{
    int number;

public:
    // Default Constructor
    Derived()
    {
        number=0;
        cout<<"Default Constructor Called"<<endl;
    }

    // Parameterized Constructor
    Derived(int n)
    {
        number=n;
    }
}
```

```
    cout<<"Parameterized Constructor Called"<<endl;
}

// Copy Constructor
Derived(Derived &obj)
{
    number=obj.number;
    cout<<"Copy Constructor Called"<<endl;
}

void display()
{
    cout<<"Number = "<<number<<endl;
}

~Derived()
{
    cout<<"Derived Destructor Called"<<endl;
}
};

int main()
{
    cout<<"\nCreating Object d1\n";
    Derived d1;

    cout<<"\nCreating Object d2\n";
    Derived d2(100);

    cout<<"\nCreating Object d3\n";
    Derived d3(d2);

    cout<<"\nDisplaying Objects\n";

    d1.display();
    d2.display();
    d3.display();

    cout<<"\nProgram Ends\n";

    return 0;
}
```

10. Observation

After successful execution of the program, the following observations were made:

1. The program successfully demonstrated the use of **default constructor**, **parameterized constructor**, and **copy constructor** in C++.
2. The **default constructor** was automatically invoked when an object was created without passing any arguments.
3. The **parameterized constructor** initialized the object using user-defined values supplied during object creation.
4. The **copy constructor** successfully created a new object by copying the contents of an existing object.
5. The program demonstrated **multiple inheritance**, where the derived class inherited properties from two base classes simultaneously.
6. During object creation, constructors of the base classes were executed first, followed by the constructor of the derived class.
7. During program termination, destructors were executed automatically in the reverse order of constructor execution, i.e., the derived class destructor was executed first, followed by the destructors of the base classes.
8. The experiment clearly illustrated the automatic object initialization and cleanup mechanism provided by constructors and destructors in Object-Oriented Programming.

11. Result

The C++ program to illustrate the working of **default constructor**, **parameterized constructor**, **copy constructor**, and **destructor** was successfully implemented. The program also demonstrated **multiple inheritance** and verified the order of execution of constructors and destructors during object creation and destruction. The objectives of the experiment were achieved successfully. in

C++.

12. Conclusion

This experiment demonstrated the importance of constructors and destructors in object-oriented programming. Different types of constructors were used to initialize objects in various ways, while the destructor ensured automatic cleanup of objects when they went out of scope. The implementation of multiple inheritance showed that constructors are executed in the order of inheritance, whereas destructors are executed in the reverse order. The experiment helped in understanding object initialization, memory management, and execution flow in C++.

13. References

1. E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill Education.
2. Robert Lafore, *Object-Oriented Programming in C++*, Pearson Education.
3. Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.
4. Yashavant Kanetkar, *Let Us C++*, BPB Publications.
5. Herbert Schildt, *C++: The Complete Reference*, McGraw Hill Education.
6. GNU GCC Documentation for C++ Programming.
7. ISO C++ Standard Documentation.

14. FAQs

1. What is a constructor in C++?
2. What is the purpose of a constructor?
3. What are the different types of constructors in C++?
4. What is a default constructor?
5. What is a parameterized constructor?
6. What is a copy constructor?
7. When is a copy constructor called?
8. What is the syntax of a copy constructor?
9. What is a destructor?
10. What is the purpose of a destructor?
11. What is the syntax of a destructor?
12. What is constructor overloading?
13. What is multiple inheritance?
14. How is multiple inheritance implemented in C++?
15. In what order are constructors executed in multiple inheritance?

16. In what order are destructors executed in multiple inheritance?
17. Why are destructors executed in reverse order of constructors?
18. Can a class have more than one constructor?
19. What is the difference between a copy constructor and an assignment operator?
20. What are the advantages of using constructors and destructors in object-oriented programming?